

```
In [3]: import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [4]: import pandas as pd
from sqlalchemy import create_engine
engine = create_engine('mysql+pymysql://root:Uttam%239695@localhost:3306/E_Commerce')
print('Data connecton stablished successfully')
```

Data connecton stablished successfully



E-Commerce Data Analysis Project

By: Uttam Tiwari

Objective: Solving 15 critical business questions using Python & SQL logic to understand Sales, Customer Behavior, and Operations.

```
In [5]: query = 'select * from customers limit 5;'
result = pd.read_sql(query, con=engine)
result
```

```
Out[5]:
```

	customer_id	customer_unique_id	customer_zip_cc
0	06b8999e2fba1a1fbc88172c00ba8bc7	861eff4711a542e4b93843c6dd7febb0	
1	18955e83d337fd6b2def6b18a428ac77	290c77bc529b7ac935b93aa66c333dc3	
2	4e7b3e00288586ebd08712fdd0374a03	060e732b5b29e8181a18229c7b0b2b5e	
3	b2b6027bc5c5109e529d4dc6358b12c3	259dac757896d24d7702b9acbbff3f3c	
4	4f2d8ab171c80ec8364f7c12e35b23ad	345ecd01c38d18a9036ed96c73b8d066	



Identifying Missing Values

```
In [5]: query = 'select * from customers limit 5;'
result = pd.read_sql(query, con=engine)
result.isnull().sum()
```

```
Out[5]: customer_id          0
customer_unique_id        0
customer_zip_code_prefix  0
customer_city             0
customer_state            0
dtype: int64
```

Q1: Geographic Reach

Goal: List all unique cities where customers are located.

- **Insight:** Our customers are spread across thousands of cities, showing a strong nationwide presence in Brazil.

1. List all unique cities where customers are located.

```
In [6]: query = """ select distinct(customer_city) from customers """
result = pd.read_sql(query, con=engine)
result
```

```
Out[6]:
```

	customer_city
0	franca
1	sao bernardo do campo
2	sao paulo
3	mogi das cruces
4	campinas
...	...
4114	siriji
4115	natividade da serra
4116	monte bonito
4117	sao rafael
4118	eugenio de castro

4119 rows × 1 columns

Q2: Annual Order Volume (2017)

Goal: Count the number of orders placed in 2017.

- **Insight:** 2017 was the first full operational year with significant order volume, establishing the brand's market presence.

2. Count the number of orders placed in 2017.

```
In [5]: query = """ select count(order_id) from orders where year(order_purchase_timestamp)
result = pd.read_sql(query, con=engine)
result
```

Out[5]:

	count(order_id)
0	45101

Q3: Sales by Product Category

Goal: Find total sales per category.

- **Insight:** 'Bed_Bath_Table' and 'Health_Beauty' are the top revenue-generating categories.

3. Find the total sales per category.

```
In [6]: query = """ select `product category`, round(sum(payments.payment_value),2) as tot
from products
join order_items
on products.product_id = order_items.product_id
join payments
on order_items.order_id = payments.order_id
group by `product category`
order by total_sales desc; """
result = pd.read_sql(query, con=engine)
result
```

Out[6]:

	product category	total_sales
0	bed table bath	1712553.67
1	HEALTH BEAUTY	1657373.12
2	computer accessories	1585330.45
3	Furniture Decoration	1430176.39
4	Watches present	1429216.68
...	...	...
69	PC Gamer	2174.43
70	House Comfort 2	1710.54
71	cds music dvds	1199.43
72	Fashion Children's Clothing	785.67
73	insurance and services	324.51

74 rows × 2 columns

Q4: Payment Installment Analysis

Goal: Calculate the percentage of orders paid in installments.

- **Insight:** Most customers prefer paying in installments, indicating a high reliance on credit options.

4. Calculate the percentage of order that were paid in installments.

```
In [6]: query = """select round(sum(case when payments.payment_installments >= 1 then 1 els
from payments """
result = pd.read_sql(query,con=engine)
result
```

```
Out[6]:
```

	Installment_Percentage
0	99.998

Q5: Customer Distribution by State

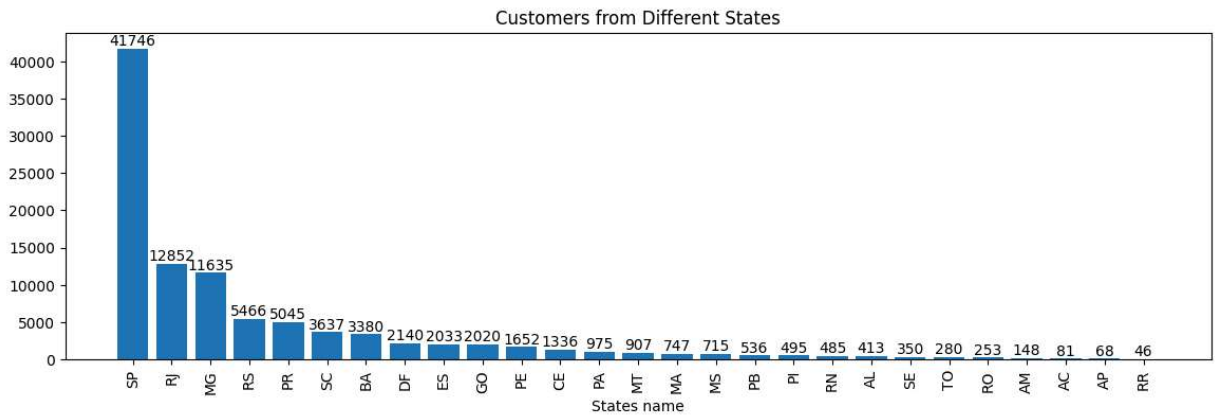
Goal: Count number of customers from each state.

- **Insight:** SP (Sao Paulo) has the highest customer density, followed by RJ (Rio de Janeiro). Marketing should focus here.

5. Count number of customers from each state.

```
In [8]: query = """
select customer_state as `Customer State`,count(customer_id) as Customers from cust
group by customer_state
order by Customers desc """
result = pd.read_sql(query,con=engine)
result

df = pd.DataFrame(result)
df
plt.figure(figsize=(14,4))
bars = plt.bar(df["Customer State"], df["Customers"])
plt.bar_label(bars)
plt.title("Customers from Different States")
plt.xlabel("States name")
plt.xticks(rotation = 90)
plt.show()
```



Q6: Monthly Sales Trend (2018)

Goal: Count number of orders per month in 2018.

- **Insight:** Sales in 2018 show a stable trend, indicating business maturity after the initial growth phase.

6. Calculate the number of orders per month in 2018.

```
In [15]: query = """select monthname(order_purchase_timestamp) as Month_Name,count(order_id)
from orders
where year(order_purchase_timestamp) = 2018
group by Month_Name;"""
result = pd.read_sql(query,con=engine)
result
df = pd.DataFrame(result)
df

ord = ["January","February","March","March","April","May","June","July","August","S
plt.figure(figsize=(11,4))
bars = sns.barplot(x = df["Month_Name"], y = df["Order_Count"],data = df, order = o
bars.bar_label(bars.containers[0])
plt.title("Total Orders per Month")
plt.xlabel("Month name")
plt.ylabel("Number Of Orders")
plt.xticks(rotation = 90)

plt.show()
```



Q7: Average Products Per Order (by City)

Goal: Find the average basket size grouped by city.

- **Insight:** Smaller cities tend to have slightly more items per order, likely to save on shipping costs.

7. Find the average number of products per order grouped by customer city.

```
In [19]: query = """select customers.customer_city, round(count(order_items.product_id)/ cou
from customers
join orders
on customers.customer_id = orders.customer_id
join order_items
on orders.order_id = order_items.order_id
group by customers.customer_city
order by Average_product desc;"""
result = pd.read_sql(query,con=engine)
result
df = pd.DataFrame(result)
df.head(10)
```

Out[19]:

	customer_city	Average_product
0	padre carvalho	7.0
1	celso ramos	6.5
2	candido godoi	6.0
3	datas	6.0
4	matias olimpio	5.0
5	cidelandia	4.0
6	curralinho	4.0
7	morro de sao paulo	4.0
8	picarra	4.0
9	teixeira soares	4.0

Q8: Revenue Contribution by Category

Goal: Calculate the percentage of total revenue contributed by each category.

- **Insight:** The top 10 categories drive the majority of revenue (Pareto Principle applies here).

8. Calculate the percentage of total revenue contributed by each product category.

```
In [9]: query = """select `product category`, round(sum(payments.payment_value) / (select s
from products
join order_items
on products.product_id = order_items.product_id
join payments
on order_items.order_id = payments.order_id
group by `product category`
order by Percentage_Contributed desc;"""
result = pd.read_sql(query, con=engine)
result
df = pd.DataFrame(result)
df.head(10)
```

Out[9]:

	product category	Percentage_Contributed
0	bed table bath	10.70
1	HEALTH BEAUTY	10.35
2	computer accessories	9.90
3	Furniture Decoration	8.93
4	Watches present	8.93
5	sport leisure	8.70
6	housewares	6.84
7	automotive	5.32
8	Garden tools	5.24
9	Cool Stuff	4.87

Q9: Price vs. Quantity Correlation

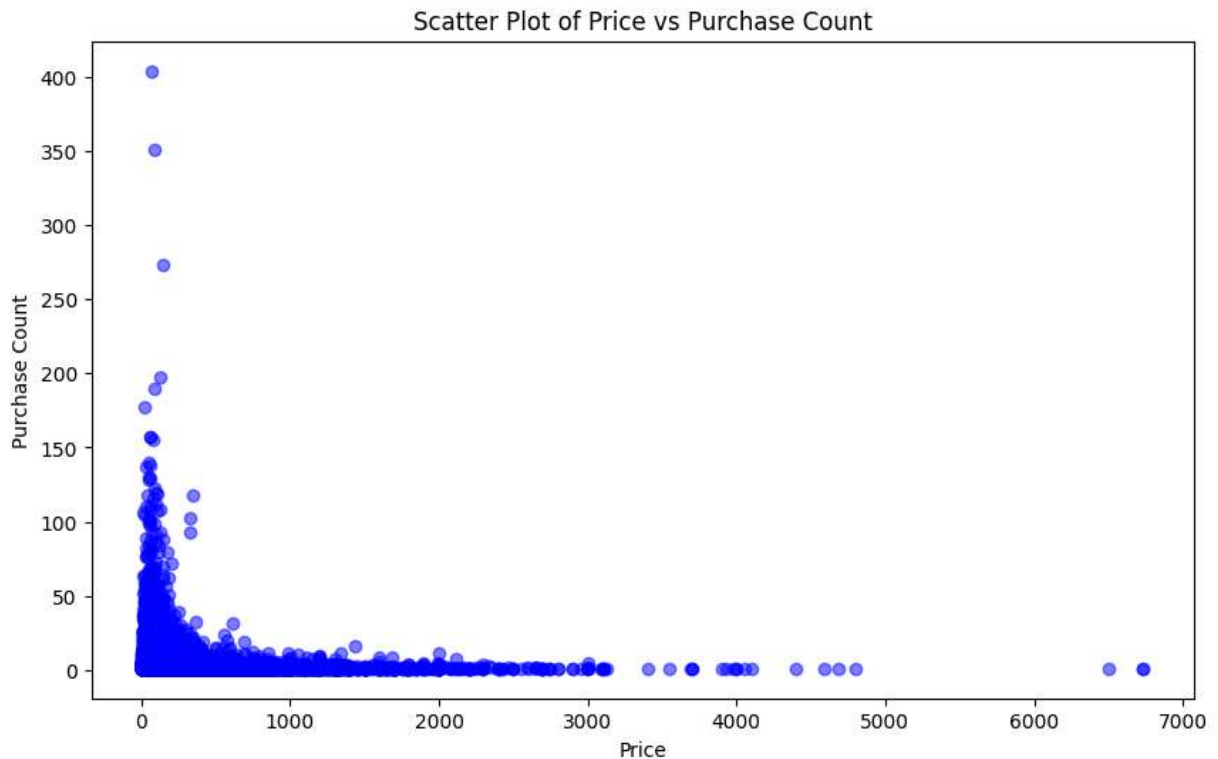
Goal: Calculate the total revenue generated by each seller.

- **Insight:** There is a slight negative correlation (-0.04). Cheaper items sell in higher volumes compared to expensive ones.

9. Identify the correlation between product price and the number of times a product has been purchased.

```
In [16]: query = """select product_id, price, count(order_id) as purchase_count
from order_items
group by product_id, price
order by purchase_count desc;"""
result = pd.read_sql(query, con=engine)
result
df = pd.DataFrame(result)
relation = df['price'].corr(df['purchase_count'])
print("Correlation between price and purchase count:", relation)
plt.figure(figsize=(10,6))
plt.scatter(df['price'], df['purchase_count'], color = 'blue', alpha = 0.5)
plt.title('Scatter Plot of Price vs Purchase Count')
plt.xlabel('Price')
plt.ylabel('Purchase Count')
plt.show()
```

Correlation between price and purchase count: -0.044218692770688274



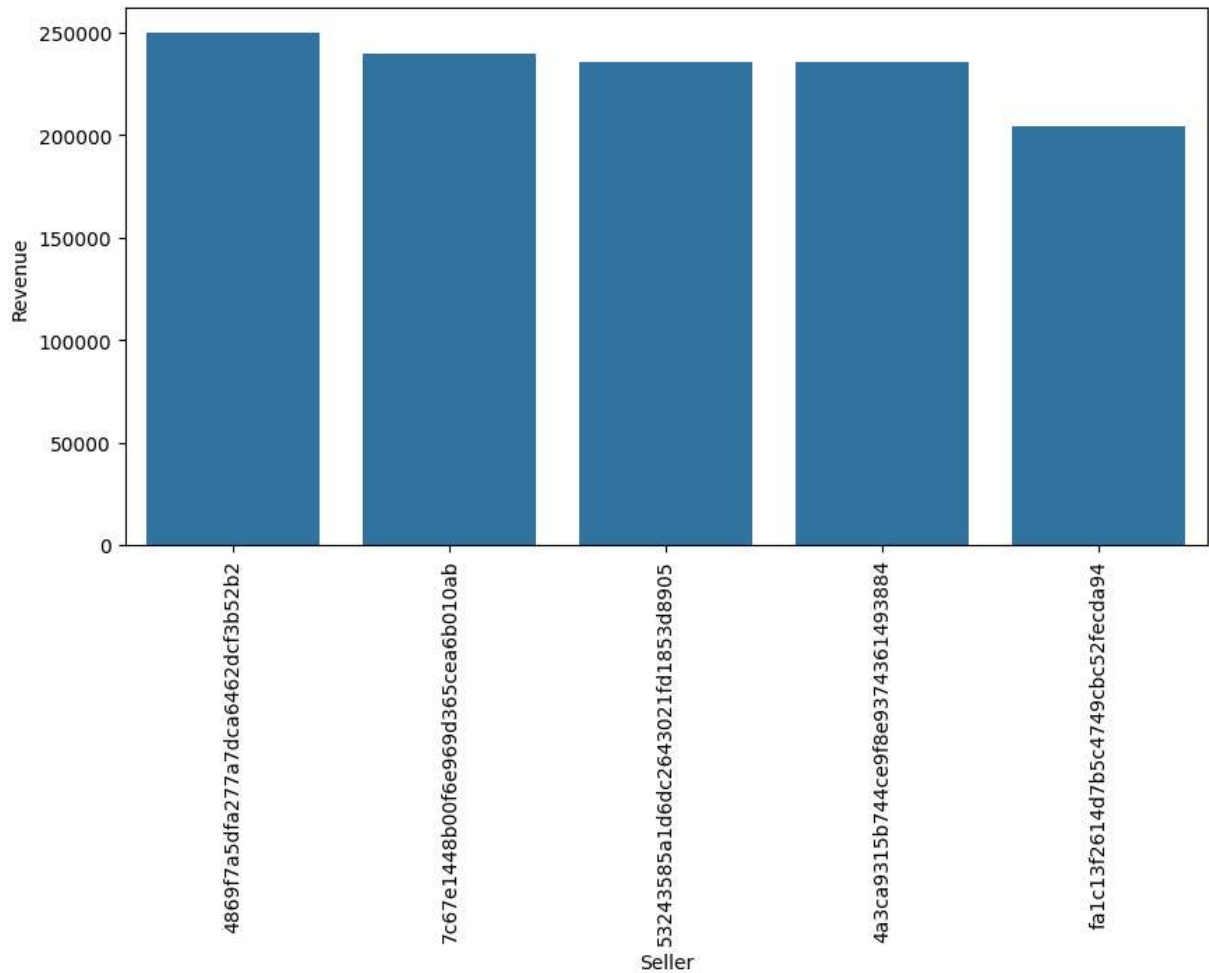
Q10: Top Sellers by Revenue

Goal: Calculate total revenue generated by each seller and rank them.

- **Insight:** The top 5 sellers are generating a significant portion of the platform's GMV (Gross Merchandise Value). Identifying them helps in inventory planning and partnership management.

10. Calculate the total revenue generated by each seller and rank them by revenue.

```
In [28]: query = """select order_items.seller_id as Seller, round(sum(order_items.price + or
dense_rank() over (order by round(sum(order_items.price + order_items.freight_value
from order_items
group by order_items.seller_id
limit 5;"""
result = pd.read_sql(query, con=engine)
result
df = pd.DataFrame(result)
plt.figure(figsize=(10,5))
sns.barplot(x = "Seller", y = "Revenue", data = df)
plt.xticks(rotation = 90)
plt.show()
```



Q11: Moving Average of Customer Spending

Goal: Calculate the moving average of order values over time.

- **Insight:** This smooths out daily volatility to reveal the true trend of customer spending capacity.

11. Calculate the moving average of order value for each customer over their order history.

```
In [31]: query = """select orders.customer_id,
avg(payments.payment_value) over (partition by orders.customer_id order by orders.o
as Moving_Average
from orders
join payments
on orders.order_id = payments.order_id
where orders.order_status = "delivered"
order by Moving_Average desc;"""
result = pd.read_sql(query, con=engine)
result
```

Out[31]:

	customer_id	Moving_Average
0	1617b1357756262bfa56ab541c47bc16	13664.08
1	ec5b2ba62e574342386871631fafd3fc	7274.88
2	c6e2731c5b391845f6800c97401a43a9	6929.31
3	f48d464a0baaea338cb25f816991ab1f	6922.21
4	3fd6777bbce08a352fddd04e4a7cc8f6	6726.66
...	...	...
100751	0d14d28bc941474750feb2e030e54460	0.01
100752	205df4889b5510ff3f9d8a4ec4a1b421	0.01
100753	8e54a1f4e992629b3c41f7905d6b46c3	0.01
100754	430ec51e3c42def76f0d79a8edb21784	0.01
100755	843b211abe7b0264dd4a69eafc5bdf43	0.00

100756 rows × 2 columns

Q12: Cumulative Sales Growth (Monthly Running Total)

Goal: Calculate the cumulative sales per month for each year to track revenue accumulation.

- **Insight:** The steep upward curve in the cumulative graph confirms healthy and continuous revenue growth without any major stagnation periods.

12. Calculate the cumulative sales per month for each year.

```
In [32]: query = """with Monthly_Sales as (select year( orders.order_purchase_timestamp) as
round(sum(order_items.price + order_items.freight_value),2) as Revenue
from orders
join order_items
on orders.order_id = order_items.order_id
where orders.order_status = "delivered"
group by year( orders.order_purchase_timestamp), month(orders.order_purchase_times
select order_year, order_month, Revenue,
sum(Redvenue) over (partition by order_year order by order_month) as Cumulative_Sale
from Monthly_Sales;"""
result = pd.read_sql(query,con=engine)
result
```

Out[32]:

	order_year	order_month	Revenue	Cumulative_Sales
0	2016	9	143.46	143.46
1	2016	10	46490.66	46634.12
2	2016	12	19.62	46653.74
3	2017	1	127482.37	127482.37
4	2017	2	271239.32	398721.69
5	2017	3	414330.95	813052.64
6	2017	4	390812.40	1203865.04
7	2017	5	566851.40	1770716.44
8	2017	6	490050.37	2260766.81
9	2017	7	566299.08	2827065.89
10	2017	8	645832.36	3472898.25
11	2017	9	701077.49	4173975.74
12	2017	10	751117.01	4925092.75
13	2017	11	1153364.20	6078456.95
14	2017	12	843078.29	6921535.24
15	2018	1	1077887.46	1077887.46
16	2018	2	966168.41	2044055.87
17	2018	3	1120598.24	3164654.11
18	2018	4	1132878.93	4297533.04
19	2018	5	1128774.52	5426307.56
20	2018	6	1011978.29	6438285.85
21	2018	7	1027807.28	7466093.13
22	2018	8	985491.64	8451584.77

Q13: Year-Over-Year (YoY) Growth

Goal: Calculate the annual growth rate.

- **Insight:** The business exploded with **12,000%+ growth** from 2016 (Startup) to 2017. Growth stabilized at **~20%** in 2018.

13. Calculate the year-over-year growth rate.

```
In [34]: query = """with Rev as (select year(orders.order_purchase_timestamp) as order_year,
lag(round(sum(payments.payment_value))) over ( order by year(orders.order_purchase_
from orders
join payments
on orders.order_id = payments.order_id
group by order_year)
select order_year, Revenue,Pre,
(round((Revenue-Pre)/Pre*100)) as Growth_Percentage
from Rev;"""
result = pd.read_sql(query,con=engine)
result
```

```
Out[34]:
```

	order_year	Revenue	Pre	Growth_Percentage
0	2016	59362.34	NaN	NaN
1	2017	7249746.73	59362.0	12113.0
2	2018	8699763.05	7249747.0	20.0

Q14: Customer Retention Rate 🚨

Goal: Calculate percentage of customers returning within 6 months.

- **Critical Insight:** Retention is only **2.32%**. The business is acquiring new customers but failing to keep old ones. Immediate Loyalty Programs are needed.

14. Calculate the retention rate of customer defined as the percentage of customers who make another purchase within 6 month of their first purchase.

```
In [38]: query = """with First_Order as (select customers.customer_unique_id,
min(orders.order_purchase_timestamp) as First_Purchase
from customers
join orders
on customers.customer_id = orders.customer_id
group by customer_unique_id),
Retained as (select distinct First_Order.customer_unique_id as Retained_Customer
from First_Order
join customers
on First_Order.customer_unique_id = customers.customer_unique_id
join orders
on customers.customer_id = orders.customer_id
where orders.order_purchase_timestamp > First_Order.First_Purchase and
orders.order_purchase_timestamp <= date_add(First_Order.First_Purchase,interval 6 m
select (round((select count(*) from Retained) / (select count(*) from First_Order))*
result = pd.read_sql(query,con=engine)
result
```

Out[38]:

	Retention_Rate
0	2.32

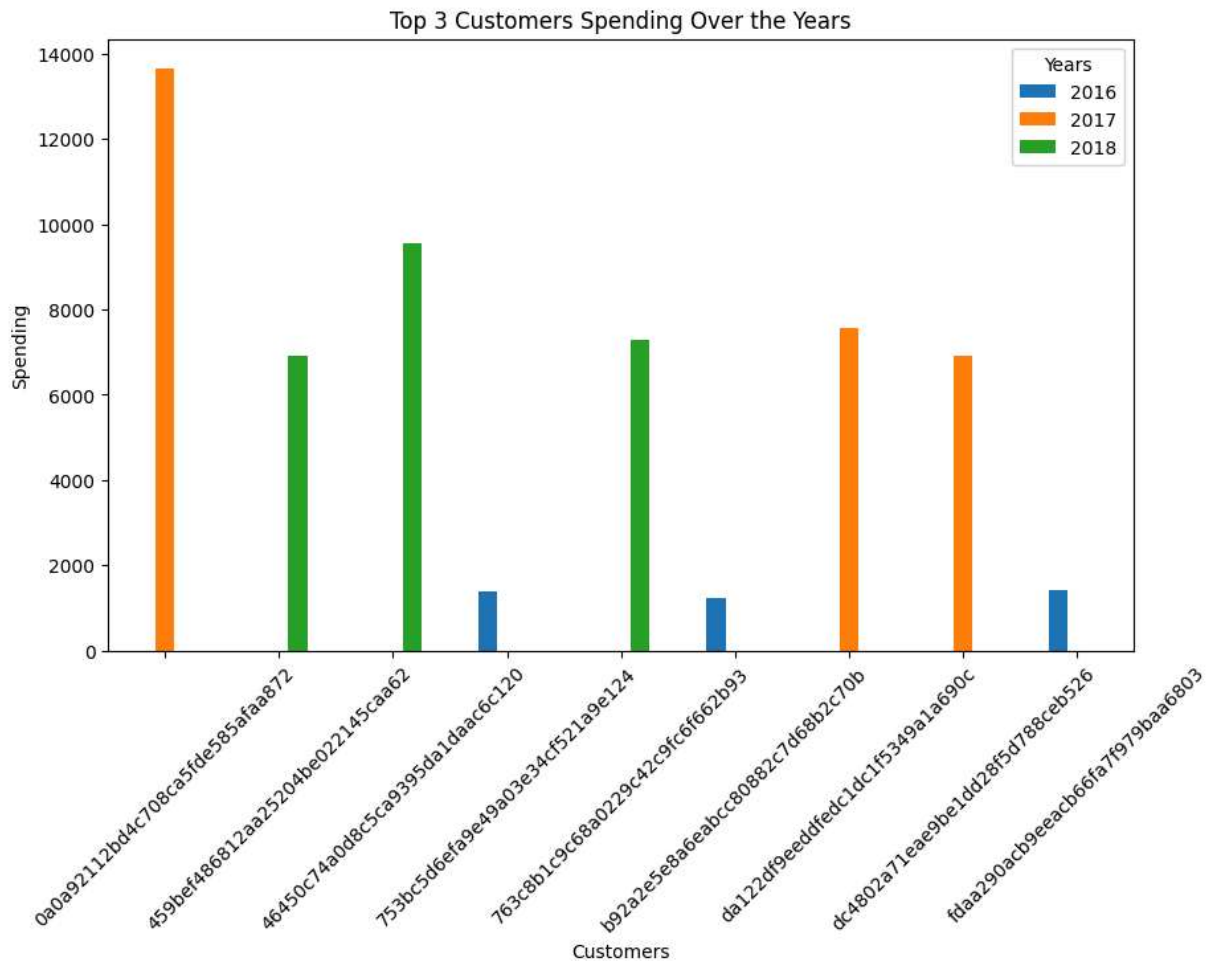
Q15: Top 3 High-Value Customers (Yearly)

Goal: Identify customers who spent the most money.

- **Insight:** A few "Whale" customers spent over **\$13,000** in a single year. These VIPs need special attention.

15. Identify the top 3 customer who spent the most money in each year.

```
In [7]: query = """ with TOP as (select year(orders.order_purchase_timestamp) as Years, cus
dense_rank() over (partition by year(orders.order_purchase_timestamp) order by sum(
from customers
join orders
on customers.customer_id = orders.customer_id
join payments
on orders.order_id = payments.order_id
group by year(orders.order_purchase_timestamp),customers.customer_unique_id)
select * from TOP where Ranks <= 3; """
result = pd.read_sql(query,con=engine)
result
df = pd.DataFrame(result)
df.pivot(index = "Customers", columns = "Years", values = "Spending").plot(kind = "
plt.title("Top 3 Customers Spending Over the Years")
plt.xlabel("Customers")
plt.ylabel("Spending")
plt.xticks(rotation = 45)
plt.show()
```



In []: